Git & GitHub

Clean Notes + Interview Q&A

PART 1

Part 1.1

Meaning · Why they exist · Real-world analogies

1. What Git is — short, precise definition

Git is a distributed version control system (DVCS) designed to track changes in files and coordinate work between multiple developers.
Technically, Git records a project's history as a sequence of *snapshots* (commits). Each snapshot is content-addressed (identified by a SHA-1/SHA-256 hash) and stored in a local repository. Because every developer has a full copy of the repository, Git supports offline work, fast operations, and branching as a first-class concept.

Key properties:

- **Distributed** — every clone contains full history.
- **Content-addressable** — commits and objects are identified by hashes.
- **Snapshot model** — commits are snapshots of the project state, not a list of diffs.
- **Branching & merging** — cheap and fast, encourages isolated workstreams.

2. What GitHub is — exact role

GitHub is a remote hosting platform and collaboration service built on top of Git. It provides:

- Remote repository storage (push/pull)
- Collaboration primitives (forks, pull requests, code review)
- Project tools (issues, project boards, wikis)
- Automation & CI/CD (Actions)
- Package registries and release management

Important distinction:

- **Git = tool / protocol / local system**
- **GitHub = cloud platform + web UX + extra services**

Git works without GitHub; GitHub cannot operate without Git repositories underneath.

3. The problem Git solves — everyday developer pain points

Before VCS (or without using it correctly), teams face repeated issues:

- Overwriting each other's files (concurrent edits)
- Losing history of why and when changes were made
- Inability to revert reliably to a previous stable state
- Difficulty integrating work from multiple people
- Poor traceability for bug introduction and fixes

Git solves these with:

- **Atomic commits** that record a state and message
- **Branch isolation** to develop features independently
- **History and metadata** (author, timestamp, message, SHA)
- **Efficient merging** to combine branches
- **Local repositories** enabling offline commits and local experimentation

4. Core mental model — how to think about Git

Think of Git as **a highly optimized time-travel filesystem** that stores lightweight snapshots and links them together. The main mental objects:

- **Working Directory**: the files you see and edit.
- **Staging Area (Index)**: a prepared set of changes queued for the next snapshot.
- **Commit (Object)**: an immutable snapshot with metadata (author, time, message, parent SHA).
- **Branch**: a movable pointer to a commit (a named reference to a snapshot).
- **HEAD**: the current branch pointer — where new commits land.
- **Remote**: a pointer to an external repository (e.g., origin on GitHub).

Flow (conceptually):

Working Directory -> Stage (Index) -> Commit -> (local history)

↓

Push -> Remote (GitHub)

5. Why distributed (not centralized) matters

Two common models exist historically: centralized VCS (e.g., SVN) and distributed VCS (Git).

Advantages of Git's distributed model:

- **Offline commits**: you can commit and inspect history without network access.
- **Performance**: local operations (log, diff, branch) are fast because data is local.
- **Resilience**: full history exists on every clone; no single point of failure.
- **Flexible workflows**: supports many branching models (centralized, feature-branch, forking, etc.).

Trade-offs (be aware):

- Distributed model means more responsibility for syncing and understanding remote vs local state.
- Requires understanding of push/pull/fetch to avoid unexpected divergence.

6. Real-world analogies (technical, precise)

Use analogies that map directly to Git concepts — they help build intuition without oversimplifying.

Analogy A — Project as a Book (best for history + branching)

- **Repository** = a folder with all book drafts.
- **Commit** = a saved edition of the book (finalized snapshot).
- **Branch** = a separate draft for a new chapter or alternate ending.
- **Merge** = combining a chapter from one draft into the main edition, with possible edits to resolve conflicts.
- **Rebase** = rewrite your draft so it appears you wrote it on top of the current main edition (changes history).

Why useful: helps understand immutable commits and branching as parallel drafts.

Analogy B — Time-machine for files (best for checkout/revert)

- **Working Directory** = present time (current working copy).
- **Commit** = a checkpoint saved in the time-machine.
- **Checkout** = go back (or forward) to a checkpoint; the filesystem updates to that state.
- **Revert/Reset** = undo changes by returning to earlier checkpoints or moving pointers.

Why useful: clarifies how commits let you travel between historical states.

Analogy C — Bank ledger + fingerprints (best for hashes & immutability)

- **Commit hash** = transaction ID generated from content (like a fingerprint).
- **Parent links** = pointers to previous transactions.
- **Immutability** = once a transaction (commit) is created, changing its content changes its ID — you can detect tampering.

Why useful: explains why the SHA identifies exact content and why rewriting history is not to be done lightly.

Analogy D — Staging area as a shipping container

- **Working files** = items produced in factory.
- **Staging area** = packing items into shipping container for the next shipment.
- **Commit** = the sealed container sent to storage.
- **Push** = sending sealed container to remote warehouse (GitHub).

Why useful: clarifies why staging exists as an intermediate step before creating a commit.

7. Common workflows (high-level, without commands)

Single developer (local-first)

1. Edit files locally.
2. Stage changes to build a coherent snapshot.

3. Commit frequently with clear messages.
4. Push to remote periodically for backup.

Team using central branch (main/master)

1. Create feature branch from main.
2. Develop and commit on feature branch.
3. Push branch and open a Pull Request (PR) on GitHub.
4. Code review, CI runs, merge PR into main.
5. Delete feature branch.

Open-source / Forking model

1. Fork upstream repo on GitHub.
2. Clone fork locally, create a branch.
3. Work and push to fork.
4. Open PR from fork to upstream repository.
5. Maintainers review and merge.

These workflows map to different collaboration needs but share the same Git fundamentals: snapshots, branching, and syncing with remotes.

8. Practical reasons Git + GitHub is the default in industry

- **Tooling ecosystem**: IDE integrations, CI/CD, code review, package registries.
- **Standardized collaboration**: PRs + reviews enforce code quality and auditing.
- **Traceability**: every commit links work to an author and a message—useful for debugging and accountability.
- **Community & discoverability**: GitHub hosts open-source projects, documentation, and examples that are reusable.

9. Mistakes to avoid from the conceptual side (preview)

(Details and recovery commands will be in later parts.)

- Treating commits as a backup without meaningful messages.
- Doing large commits that mix unrelated changes.
- Confusing local state with remote state (push/pull semantics).

- Rebasing public/shared history without coordination.

10. Summary — what to retain from Part 1.1

- Git = distributed version control; stores snapshots locally; uses hashes for integrity.
- GitHub = remote hosting + collaboration tooling layered on top of Git.
- Mental model: working directory → staging area → commit → branches → remote.
- Use analogies (book drafts, time-machine, ledger, shipping container) to build intuition.
- Different workflows exist (single developer, centralized team, fork & PR); all rely on the same core primitives.

Part 1.2

Deep Mental Model · Git Internals · How Git Actually Stores Data

1. Why understanding Git internals matters

Most beginners use Git as a **set of commands**.
Engineers understand Git as a **data model**.

If you understand:

- what Git stores
- where it stores it
- how references move

then:

- commands stop feeling magical
- mistakes become recoverable
- advanced workflows make sense

This section builds that **internal mental model**.

2. Git is NOT a file-diff system (common misconception)

Many assume:

"Git stores differences between files"

This is **incorrect**.

What Git actually stores

Git stores:

- **snapshots of the entire project state**
- optimized internally using content-addressed storage

Each commit represents:

"This is exactly how the project looked at this moment"

Internally, Git may compress data, but **conceptually** it is snapshot-based.

3. The .git directory — Git's brain

When you run:

git init

Git creates a hidden directory:

.git/

This directory contains:

- entire project history
- commits
- branches
- configuration
- references

Deleting .git = deleting Git itself
Project files remain, history is gone.

4. Core Git object types (very important)

Git stores everything as **objects**.
There are **four fundamental object types**.

4.1 Blob (Binary Large Object)

A **blob**:

- stores file **content only**
- does NOT store filename
- does NOT store directory structure

Example:

- main.c content → blob
- README.md content → blob

If two files have identical content:

- Git stores **only one blob**

- both files reference it

4.2 Tree

A **tree**:

- represents a directory
- maps **names → blobs or other trees**

Tree stores:

- filenames
- directory structure
- pointers to blobs/trees

Think of tree as:

"Folder metadata + references"

4.3 Commit

A **commit** object stores:

- reference to a root tree
- reference to parent commit(s)
- author and committer info
- timestamp
- commit message

Important properties:

- Immutable
- Identified by a hash
- Forms a directed graph (history)

A commit does **not** store files directly.
It stores a pointer to a tree.

4.4 Tag (brief mention)

A **tag**:

- human-readable reference to a commit
- often used for releases (v1.0.0)

We'll revisit tags later.

5. Commit graph — how history is formed

Each commit points to:

- its parent commit
- or multiple parents (merge commit)

This forms a **Directed Acyclic Graph (DAG)**.

Example:

C1 → C2 → C3

↘

C4 (merge)

Important implications:

- History is not a straight line
- Branching is natural
- Merges create multi-parent commits

6. What a branch REALLY is

Common misunderstanding

"A branch is a copy of code"

Wrong.

Reality

A **branch is just a pointer** to a commit.

Example:

main → C3

feature-x → C2

When you commit:

- Git moves the branch pointer forward

- Commits themselves never move

This is why:

- creating branches is cheap
- switching branches is fast

7. HEAD — the most misunderstood concept

HEAD is a pointer to:

- the currently checked-out branch
- or directly to a commit (detached HEAD)

Normally:

HEAD → main → C3

Detached HEAD:

HEAD → C2

In detached HEAD:

- commits are possible
- but not attached to a branch
- easy to lose if not referenced

8. Three core states of files (daily confusion source)

Every file tracked by Git exists in **one of three states**:

8.1 Working Directory

- Actual files on disk
- You edit these

8.2 Staging Area (Index)

- Files prepared for next commit
- Snapshot-in-progress

8.3 Repository (Committed)

- Files stored permanently in history

Flow:

Edit → Stage → Commit

This separation is intentional and powerful.

9. Why the staging area exists

Without staging:

- every commit would include all changes
- impossible to group logical changes

With staging:

- you choose **what goes into the next commit**
- supports clean, atomic commits

Example:

- Fix bug + refactor code
- Stage only bug fix
- Commit
- Stage refactor
- Commit separately

This is professional Git usage.

10. Local repository vs Remote repository

Local repository

- Exists on your machine
- Full history
- Full commit graph

Remote repository (e.g., GitHub)

- Another copy of the same history
- Used for collaboration and backup

Important:

- Git does NOT auto-sync
- Push and pull are explicit actions

11. Remote references (conceptual)

When you add a remote:

origin/main

This means:

- "My last known state of main on origin"

It is:

- a **read-only pointer**
- updated only when you fetch or pull

This prevents accidental overwrites.

12. Fetch vs Pull (conceptual, no commands yet)

Fetch

- Updates remote-tracking branches
- Does NOT change working directory

Pull

- Fetch + merge (or rebase)
- Changes working directory

Understanding this distinction prevents many errors.

13. How Git ensures data integrity

Git uses:

- cryptographic hashes (SHA)

Hash is computed from:

- object content
- object type
- metadata

Result:

- Changing content changes hash

- History tampering is detectable
- Objects are immutable

This is why Git is reliable.

14. What happens when you commit (step-by-step)

Internally, Git:

1. Converts staged files into blobs
2. Creates trees for directory structure
3. Creates commit object pointing to tree
4. Moves branch pointer to new commit
5. Updates HEAD

No magic. Just pointers and objects.

15. Summary — mental model checkpoint

By now, you should understand:

- Git stores objects, not diffs
- Commits point to trees, trees to blobs
- Branches are movable pointers
- HEAD determines current context
- Staging area enables atomic commits
- Local and remote repositories are independent

This mental model is **critical** before learning commands.

Part 1.3

Git Lifecycle · First Repository · Core Commands (Explained from Inside)

1. From theory to practice: how Git is actually used

Until now, you learned:

- what Git is
- how Git stores data internally
- how commits, branches, and HEAD work

Now we connect this **mental model** to **real usage**.

This part explains:

- how a Git repository is created
- how files move through Git's lifecycle
- what core commands do **internally**, not just syntactically

2. Creating a Git repository

There are two ways a repository comes into existence.

2.1 Initializing a new repository

When you run:

git init

Git:

- creates a .git directory
- sets up default configuration
- creates an empty object database
- sets HEAD to point to a default branch (usually main)

At this point:

- No commits exist
- No files are tracked
- History is empty

Your project is now **Git-aware**, not yet versioned.

2.2 Cloning an existing repository

When you run:

git clone <repository-url>

Git:

- downloads the entire commit history
- copies all branches
- sets up a remote called origin
- checks out the default branch

This is why clones are large but powerful:

- You get **full history**
- You can work offline immediately

3. File lifecycle in Git (very important)

Every file in a Git repository moves through well-defined states.

3.1 Untracked

- File exists in working directory
- Git does not know about it
- Not part of any commit

Example:

- New file created

3.2 Tracked

A file becomes tracked once it is added to Git.

Tracked files can be:

- **Unmodified**
- **Modified**
- **Staged**

3.3 Modified

- File content changed
- Git knows it has changed
- Changes are not staged yet

3.4 Staged

- File snapshot added to staging area
- Ready to be included in next commit

3.5 Committed

- File snapshot stored permanently in repository
- Part of history

Lifecycle summary:

Untracked → Modified → Staged → Committed

4. git status — the most important command

git status

This command:

- inspects working directory
- inspects staging area
- compares both with last commit

It answers:

- What changed?
- What is staged?
- What is untracked?

Internally:

- Git compares file hashes
- Determines file state

Rule:

If confused, run git status.

5. git add — misunderstood command

Common misconception

“git add saves the file”

Incorrect.

Reality

git add:

- copies the current file snapshot
- into the staging area (index)

git add file.txt

What happens internally:

- Git creates a blob for file content
- Updates staging area to point to that blob

Important:

- You can modify the file again after staging
- Staged version and working version can differ

5.1 git add .

git add .

- Adds all modified and untracked files
- Except ignored files
- Stages snapshots, not filenames

Use carefully in large projects.

6. Staging area vs Working directory (practical view)

Scenario:

1. Edit file A

2. Edit file B
3. Stage file A only
4. Commit

Result:

- Commit contains changes from file A
- File B changes remain unstaged

This is intentional and powerful.

7. git commit — creating history

git commit -m "message"

A commit:

- creates a new commit object
- points to staged tree
- records metadata
- moves branch pointer forward

Internally, Git:

1. Writes blobs for staged files
2. Creates tree structure
3. Creates commit object
4. Updates branch reference
5. Moves HEAD

Commits are:

- immutable
- content-addressed
- permanent (unless history is rewritten)

8. Commit messages (technical importance)

Commit messages are:

- not comments

- not decoration
- part of project documentation

Good commit messages:

- explain **why**, not just **what**
- describe one logical change

Bad:

update

fix

changes

Good:

Fix null pointer check in user authentication

9. git log — reading history

git log

Displays:

- commit hash
- author
- date
- message

This is reading the commit graph.

Variants exist:

- compact views
- one-line history
- graphical views

Understanding log output requires understanding:

- parent-child relationships
- branch pointers

10. git diff — inspecting changes

git diff compares:

- working directory vs staging area
- staging area vs last commit

This answers:

- What exactly changed?
- What will be committed?

Internally:

- Git compares blob content hashes
- Shows line-level differences

11. Ignoring files — .gitignore

Some files should never be tracked:

- build outputs
- dependency folders
- secrets
- OS-specific files

.gitignore tells Git:

- which files to ignore
- before staging

Ignored files:

- remain untracked
- never enter history

Important:

- .gitignore must be committed
- Ignoring works only for untracked files

12. What happens if you forget to stage?

If you commit without staging:

- Git refuses
- No commit is created

This enforces:

- deliberate snapshots
- controlled history

13. Common beginner mistakes (conceptual)

- Editing files and forgetting to stage
- Staging everything blindly
- Writing vague commit messages
- Treating commits as backups
- Ignoring git status

Understanding lifecycle prevents these.

14. Practical workflow (local-only)

1. Create or modify files
2. Check status
3. Stage selected changes
4. Review diff
5. Commit
6. Repeat

This loop forms the base of all Git usage.

15. Summary — Part 1.3 checkpoint

You should now understand:

- How repositories are created
- File states and transitions
- What add, commit, status, log, and diff really do
- Why staging exists
- How commits form history

This is the **minimum foundation** before collaboration.

Part 1.4

Branching from Zero · Creating, Switching, Merging · How Branches Actually Move

1. Why branching exists (engineering reason)

In real software development:

- Multiple features are developed in parallel
- Bug fixes must not break stable code
- Experiments should not affect production-ready code

Without branching:

- Everyone works on the same code
- Conflicts increase
- Stability drops

Branching solves isolation.

A branch allows you to:

- Work independently
- Keep main code stable
- Merge only when ready

2. The most important clarification (again)

A branch is NOT a copy of code.

A branch is:

- A **named pointer** to a commit
- A lightweight reference
- Extremely cheap to create

This single fact explains:

- Why branches are fast
- Why switching branches is instant

- Why Git scales well

3. Visualizing branches (mental model)

Assume this commit history:

C1 → C2 → C3

Pointers:

main → C3

HEAD → main

Now create a new branch:

feature-login → C3

Nothing is duplicated.
Only a new pointer is created.

4. Creating branches

Command

git branch feature-login

What Git does internally:

- Creates a new reference
- Points it to the current commit
- Does NOT switch to it

After this:

main → C3

feature-login → C3

5. Switching branches (checkout / switch)

Command

git checkout feature-login

(or newer syntax)

git switch feature-login

Internally:

- HEAD moves to feature-login
- Working directory updates to match that commit

Now:

HEAD → feature-login → C3

Any new commit will move **feature-login**, not main.

6. Making commits on a branch

After switching:

- Edit files
- Stage
- Commit

Result:

C1 → C2 → C3 → C4

↑

feature-login

main → C3

Key observation:

- Branch pointers diverge
- Commits remain immutable
- Only pointers move

7. Switching back to main

git checkout main

Now:

HEAD → main → C3

feature-login → C4

Working directory updates automatically.

This is why:

- Uncommitted changes block switching
- Git protects working state

8. Why uncommitted changes can block checkout

If:

- File modified on branch A
- Same file differs on branch B

Git refuses to switch to avoid:

- Overwriting work
- Data loss

Solution:

- Commit
- Or stash (covered later)

9. Merging branches (core collaboration concept)

Merging means:

Combine histories of two branches

Command

git merge feature-login

Executed **from target branch** (usually main).

10. Fast-forward merge (simplest case)

Condition:

- No new commits on main
- Feature branch is ahead

Before merge:

main → C3

feature → C4 → C5

After merge:

main → C5

feature → C5

No new commit created.
Git simply moves the main pointer forward.

11. Three-way merge (most common)

Condition:

- Both branches have new commits

Before:

C4 → C5 (feature)

/

C1 → C2 → C3 (main)

After merge:

C4 → C5

/ \

C1 → C2 → C3 → C6 (merge commit)

Git:

- Finds common ancestor (C3)
- Combines changes
- Creates a new merge commit (C6)

12. Merge commits (what they represent)

A merge commit:

- Has **two parents**
- Represents history convergence
- Preserves full branch structure

This is important for:

- Auditing
- Debugging

- Understanding development flow

13. Merge conflicts (why they happen)

A conflict occurs when:

- Same file
- Same lines
- Changed differently in two branches

Git cannot decide automatically.

Example:

- Branch A edits line 10
- Branch B edits line 10 differently

14. Resolving merge conflicts (conceptual)

Git marks conflicts inside files:

<<<<<<< HEAD

code from current branch

=======

code from merged branch

>>>>>>> feature-login

You must:

- Choose correct code
- Edit manually
- Remove markers
- Stage resolved files
- Complete merge

Conflict resolution is:

- Normal
- Expected
- Part of real development

15. Branch deletion (important cleanup)

After merge:

git branch -d feature-login

This:

- Deletes the pointer
- Does NOT delete commits
- History remains reachable through main

Git will block deletion if:

- Branch is not merged
- Risk of losing reachable commits

16. Branching strategies (high-level overview)

Common patterns:

- Feature branches
- Release branches
- Hotfix branches

Basic rule:

- main = stable
- Feature branches = temporary

Complex strategies (Git Flow, trunk-based) build on this.

17. Branching mistakes beginners make

- Working directly on main
- Creating long-lived feature branches
- Forgetting which branch is active
- Merging wrong direction
- Deleting branches too early

Understanding pointer movement prevents these.

18. Summary — Part 1.4 checkpoint

You should now clearly understand:

- What a branch actually is
- How branch pointers move
- How checkout/switch works
- Fast-forward vs merge commits
- Why conflicts occur
- How merging preserves history

Branching is **the core power of Git**.

Part 1.5

Rebase · Stash · Undoing Changes (Reset, Revert, Checkout) — Explained Calmly

1. Why this part matters

This section covers the **most feared Git concepts**:

- rebase
- stash
- undo commands

Most fear comes from **not understanding what moves and what doesn't**.

Rule to remember throughout this part:

Commits don't change. Pointers move. History can be rewritten only if you choose to.

2. Rebase — what it really means

2.1 Definition (accurate and simple)

Rebase means:

Rewriting the base of a branch by replaying its commits on top of another commit.

In simple terms:

- Take commits from branch A
- Reapply them on top of branch B
- Create new commits with new hashes

3. Rebase vs Merge — core difference

Merge

- Preserves history
- Creates a merge commit
- Branch structure remains visible

Rebase

- Rewrites history
- Produces a linear commit graph
- No merge commit

4. Visual comparison

Before

C1 → C2 → C3 (main)

\

C4 → C5 (feature)

After merge

C1 → C2 → C3 → C6

\ /

C4 → C5

After rebase

C1 → C2 → C3 → C4' → C5'

Important:

- C4' and C5' are **new commits**
- Old C4, C5 still exist but become unreachable

5. When rebase is SAFE

Rebase is safe when:

- You are working **alone**
- Branch has **not been shared**
- Commits exist only locally

Typical use:

- Clean up feature branch before merge
- Make history readable

6. When rebase is DANGEROUS

Never rebase when:

- Branch is shared with others
- Commits are already pushed and used

Why:

- Rebase changes commit hashes
- Other developers' history breaks

Rule:

Never rebase public history.

7. git rebase — what happens internally

Internally Git:

1. Finds common ancestor
2. Temporarily removes your branch commits
3. Moves branch pointer to new base
4. Replays commits one by one
5. Creates new commit objects

Understanding this removes fear.

8. Stash — temporary storage for work-in-progress

8.1 Why stash exists

Problem:

- You are working
- Need to switch branches
- Work is incomplete

Stash allows you to:

- Save current working state
- Clean working directory

- Restore later

9. What stash actually stores

Stash stores:

- Modified tracked files
- Staged changes
- Optionally untracked files

It does NOT:

- Commit anything
- Add to history

Think of stash as:

A temporary shelf, not permanent storage.

10. Typical stash workflow (conceptual)

1. Save current changes to stash
2. Switch branch
3. Do required work
4. Return to original branch
5. Reapply stash

Stashes are stack-based (LIFO).

11. Undoing changes — categories (very important)

Undoing depends on **where the change exists**.

There are three locations:

1. Working directory
2. Staging area
3. Commit history

Each requires a different approach.

12. Undo changes in Working Directory

Case:

- File modified
- Not staged
- Not committed

You want:

- Discard changes
- Restore last committed version

Effect:

- File resets to last commit state
- Data is lost (be careful)

13. Undo changes in Staging Area

Case:

- File staged
- Want to unstage
- Keep modifications

Solution:

- Remove from staging area
- Keep working directory unchanged

This is very common and safe.

14. Undo committed changes — two approaches

14.1 Reset (history-changing)

Reset:

- Moves branch pointer
- Can delete commits from current branch

Types:

- Soft: keeps changes staged

- Mixed: keeps changes unstaged
- Hard: discards changes completely

Rule:

Reset is for **local history only**.

14.2 Revert (history-safe)

Revert:

- Creates a new commit
- Undoes changes of a previous commit
- Preserves history

Used when:

- Commit already shared
- Safety matters

Rule:

Revert for public history, reset for private history.

15. Checkout vs Reset vs Revert — mental map

Action	Affects History	Safe for Shared Branch
Checkout file	No	Yes
Reset	Yes	No
Revert	No	Yes

Understanding this table prevents disasters.

16. Detached HEAD (danger zone)

Detached HEAD occurs when:

- You checkout a commit directly
- Not a branch

You can:

- View history
- Experiment
- Commit (temporarily)

Risk:

- Commits may be lost if not attached to a branch

Fix:

- Create a branch before leaving

17. Common mistakes with undo & rebase

- Rebasing after pushing
- Hard reset without checking
- Using stash as long-term storage
- Forgetting staged state
- Panicking instead of inspecting status/log

Calm inspection prevents damage.

18. Summary — Part 1.5 checkpoint

You now understand:

- What rebase truly does
- When rebase is safe vs dangerous
- Purpose of stash
- How undoing depends on file location
- Reset vs revert difference
- How to avoid panic situations

These concepts separate **casual users** from **confident Git users**.

Part 1.6

GitHub Explained Properly · Git ↔ GitHub End-to-End Workflow

1. Why GitHub exists (beyond "code storage")

Git alone solves **version control**, but not:

- Collaboration at scale
- Code review
- Access control
- Visibility
- Automation

GitHub exists to solve collaboration and coordination, not versioning itself.

GitHub adds:

- A central meeting point for repositories
- Web-based interface for history and review
- Team workflows and permissions
- Automation and integrations

2. GitHub is not "the cloud version of Git"

This misconception causes confusion.

Correct model:

- Git runs locally
- GitHub hosts **another copy** of the same repository
- Git does not automatically sync with GitHub

Sync is always **explicit**.

3. Local vs Remote — precise terminology

Local repository

- Exists on your machine

- Full history
- Full control

Remote repository

- Exists on GitHub servers
- Acts as shared reference point
- Used for collaboration and backup

They are **peers**, not master–slave.

4. Remote named origin

When you clone a repo:

- Git automatically creates a remote called origin
- origin is just a name, not special
- It points to a GitHub URL

You can have:

- multiple remotes
- multiple hosting platforms

5. Pushing changes to GitHub

What "push" actually means

Push:

- Sends commits from local repo
- Updates branch on remote
- Does not change your local history

Push only succeeds if:

- Remote branch can be fast-forwarded
- No conflicting updates exist

Push never:

- Pulls remote changes
- Resolves conflicts automatically

6. Pulling changes from GitHub

What "pull" actually does

Pull:

- Fetches changes from remote
- Integrates them into local branch
- May merge or rebase

This means:

- Your working directory can change
- Conflicts may appear

Pull should be done:

- Before starting work
- Before pushing changes

7. Fetch — the safest sync operation

Fetch:

- Downloads remote updates
- Updates remote-tracking branches
- Does NOT touch working directory

Use fetch when:

- You want to inspect remote changes
- You want control over integration

Fetch first, merge later = safer workflow.

8. Remote-tracking branches (important concept)

Example:

origin/main

This means:

- Your local view of main on GitHub

- Read-only reference
- Updated only by fetch/pull

They help Git:

- Compare local vs remote
- Prevent overwriting shared history

9. End-to-end workflow (solo developer)

1. Clone repository
2. Create local branch
3. Make commits
4. Push branch to GitHub
5. Merge locally or via PR

Even solo projects benefit from this structure.

10. Team workflow (feature branch model)

Standard industry flow:

1. Pull latest main
2. Create feature branch
3. Develop and commit
4. Push feature branch
5. Open Pull Request
6. Code review + CI
7. Merge to main
8. Delete feature branch

GitHub enforces discipline here.

11. Pull Requests (PRs) — central GitHub concept

A Pull Request is:

- A request to merge code

- A review boundary
- A discussion space

PRs enable:

- Code review
- Automated checks
- Documentation of decisions

PRs are **not optional** in serious teams.

12. What happens when a PR is opened

GitHub:

- Compares branch histories
- Shows diff
- Runs CI workflows
- Allows inline comments
- Blocks merge until conditions are met

Nothing is merged automatically.

13. Forking — open-source collaboration model

Forking is used when:

- You don't have write access
- You contribute externally

Flow:

1. Fork repo on GitHub
2. Clone your fork
3. Create branch
4. Commit changes
5. Push to fork
6. Open PR to upstream repo

Fork ≠ clone.

14. Permissions & access control

GitHub supports:

- Read
- Write
- Admin

This ensures:

- Controlled collaboration
- Accountability
- Security

Public vs private repos define visibility.

15. Issues & project tracking (brief)

GitHub Issues:

- Bug tracking
- Feature requests
- Task discussion

They integrate with:

- PRs
- Commits
- Projects

16. GitHub Actions (overview only)

GitHub Actions:

- Automates workflows
- Runs tests
- Builds and deploys code

Triggered by:

- Push

- PR
- Scheduled events

We will not go deep here (advanced topic).

17. Common GitHub mistakes

- Pushing directly to main
- Not pulling before pushing
- Ignoring PR reviews
- Treating GitHub as backup only
- Deleting branches prematurely

Understanding workflow avoids these.

18. Mental checklist before every push

- On correct branch?
- Pulled latest changes?
- Commits are clean?
- Tests passing?
- Ready for review?

This is professional discipline.

19. Summary — Part 1 (Complete)

You now understand:

- What Git and GitHub are
- Why they exist
- How Git works internally
- File lifecycle
- Branching, merging, rebasing
- Undoing safely
- How Git syncs with GitHub

- How teams collaborate using PRs

This completes **PART 1: Foundations**.

PART 2

Practical Cheat Sheet · Daily-Use Reference

1. Purpose of This Cheat Sheet

This section is designed for:

- Quick revision
- Daily development use
- Pre-interview brushing
- Debugging common Git situations

This is **not explanatory theory**.
This is **action-oriented reference material**.

2. Repository Setup

Initialize a new repository

git init

Creates a .git directory and starts version tracking.

Clone an existing repository

git clone <repository-url>

Creates a local copy with full history and sets origin.

3. Checking Repository State

Check current status

git status

Shows:

- Untracked files

- Modified files
- Staged files
- Current branch

Use this command frequently.

View commit history

git log

Compact history:

git log --oneline

Graph view:

git log --oneline --graph --all

4. File Tracking & Staging

Add specific file

git add filename

Add all changes

git add .

Remove file from staging (keep changes)

git restore --staged filename

5. Committing Changes

Create a commit

git commit -m "commit message"

Amend last commit (local only)

git commit --amend

Used to:

- Fix commit message
- Add missed files

6. Viewing Differences

Working directory vs staging

git diff

Staged vs last commit

git diff --staged

7. Branching Commands

List branches

git branch

Create new branch

git branch branch-name

Switch branch

git checkout branch-name

or

git switch branch-name

Create and switch

git checkout -b branch-name

or

git switch -c branch-name

8. Merging Branches

Merge feature branch into current branch

git merge branch-name

Fast-forward happens automatically when possible.

9. Handling Merge Conflicts (Quick Steps)

1. Identify conflicted files
2. Open file and resolve conflict markers
3. Stage resolved files
4. Complete merge commit

git add filename

git commit

10. Rebase (Local Cleanup Only)

Rebase current branch onto main

git rebase main

Abort rebase

git rebase --abort

Continue after conflict resolution

git rebase --continue

11. Stash Commands

Save current work

git stash

List stashes

git stash list

Apply last stash

git stash apply

Apply and remove stash

git stash pop

12. Undoing Changes (Quick Reference)

Discard working directory changes

git restore filename

Unstage file

git restore --staged filename

Reset last commit (local only)

Soft reset:

git reset --soft HEAD~1

Mixed reset:

git reset HEAD~1

Hard reset (dangerous):

git reset --hard HEAD~1

Revert a commit (safe for shared history)

git revert <commit-hash>

13. Remote Repository Commands

View remotes

git remote -v

Add remote

git remote add origin <url>

Push changes

git push origin branch-name

First-time push:

git push -u origin branch-name

Fetch updates

git fetch

Pull updates

git pull

14. Pull Request Workflow (GitHub)

1. Create branch locally
2. Commit changes
3. Push branch
4. Open Pull Request on GitHub
5. Review & merge
6. Delete branch

15. .gitignore Essentials

Common ignores:

node_modules/

.env

*.log

dist/

build/

Important:

- .gitignore must be committed
- Does not affect already tracked files

16. Common Errors & Fixes

Error: rejected non-fast-forward

Fix:

git pull --rebase

Accidentally committed to main

Fix:

1. Create new branch
2. Reset main
3. Push branch

Detached HEAD state

Fix:

git switch -c new-branch

17. Daily Git Workflow (Checklist)

- git status
- Edit code
- git add
- git diff
- git commit
- git pull
- git push

Repeat.

18. Summary — PART 2

This cheat sheet covers:

- 90% of daily Git usage
- Safe undo operations
- Branching and merging
- Remote sync
- Common fixes

Use this section as a **reference**, not for first-time learning.

PART 3

Complete Interview Q&A (Beginner → Advanced)

SECTION A — BASIC / ENTRY-LEVEL QUESTIONS

Q1. What is Git?

Git is a distributed version control system used to track changes in files, maintain project history, and enable collaboration between developers. It stores project history as snapshots called commits.

Q2. Why is Git used?

Git is used to:

- Track code changes
- Maintain history
- Enable collaboration
- Support branching and merging
- Prevent data loss

Q3. What is version control?

Version control is a system that records changes to files over time so that specific versions can be recalled, compared, or restored.

Q4. What is the difference between Git and GitHub?

Git is a tool used locally for version control.
GitHub is a cloud-based platform that hosts Git repositories and provides collaboration features like pull requests and code review.

Q5. Is Git dependent on GitHub?

No. Git works independently. GitHub depends on Git.

Q6. What is a repository?

A repository is a directory that Git tracks. It contains:

- Project files
- Git history
- Configuration and metadata

Q7. What is a commit?

A commit is a snapshot of staged changes along with metadata such as author, time, and message. Each commit has a unique hash.

Q8. What is a commit hash?

A commit hash is a cryptographic identifier generated from commit content and metadata. It uniquely identifies a commit.

Q9. What is the working directory?

The working directory is the folder where files are edited and modified.

Q10. What is the staging area?

The staging area is an intermediate area where selected changes are prepared before committing.

Q11. What is HEAD?

HEAD is a pointer that represents the currently checked-out branch or commit.

SECTION B — INTERMEDIATE / INTERN-LEVEL QUESTIONS

Q12. Explain the Git file lifecycle.

Files move through these states:

- Untracked
- Modified
- Staged
- Committed

Q13. What is the purpose of git status?

It shows:

- Current branch
- Modified files
- Staged files
- Untracked files

Q14. What does git add do internally?

It takes a snapshot of file content and stores it in the staging area, creating blob objects.

Q15. What does git commit do internally?

It:

- Converts staged files to blobs
- Creates tree objects
- Creates a commit object
- Moves the branch pointer

Q16. What is a branch?

A branch is a lightweight pointer to a commit that represents an independent line of development.

Q17. Why are branches cheap in Git?

Because branches are just pointers, not copies of files.

Q18. What is the difference between merge and rebase?

Merge combines histories and creates a merge commit.
Rebase rewrites history by replaying commits on top of another base.

Q19. What is a merge conflict?

A merge conflict occurs when Git cannot automatically resolve changes made to the same part of a file in different branches.

Q20. What is a fast-forward merge?

A fast-forward merge occurs when the target branch has no new commits and Git can simply move the branch pointer forward.

Q21. What is .gitignore?

.gitignore specifies files and directories that Git should not track.

Q22. Can .gitignore remove tracked files?

No. It only affects untracked files.

Q23. What is a remote?

A remote is a reference to another repository, usually hosted on a platform like GitHub.

Q24. What is origin?

origin is the default name given to the remote repository when cloning.

Q25. Difference between git fetch and git pull?

- git fetch downloads updates without changing the working directory

- git pull fetches and integrates changes

SECTION C — PRACTICAL / SCENARIO-BASED QUESTIONS

Q26. You committed to the wrong branch. What do you do?

- Create a new branch from current state
- Reset the original branch to previous commit
- Push the correct branch

Q27. You want to undo changes but keep the file. What do you use?

Unstage or restore without committing.

Q28. You want to undo a commit already pushed. What do you use?

Use git revert to create a new commit that reverses changes.

Q29. When should you use git reset --hard?

Only for local, unshared history when you want to completely discard changes.

Q30. What is a detached HEAD state?

Detached HEAD occurs when HEAD points directly to a commit instead of a branch.

Q31. How do you recover from detached HEAD?

Create a new branch before leaving the state.

Q32. What is stash used for?

Stash temporarily stores uncommitted changes so you can switch branches safely.

Q33. Can stash store untracked files?

Yes, if explicitly included.

SECTION D — GITHUB-SPECIFIC QUESTIONS

Q34. What is a Pull Request?

A Pull Request is a request to merge changes from one branch into another with review.

Q35. Why are Pull Requests important?

They enable:

- Code review
- Quality checks
- Collaboration
- Documentation of decisions

Q36. What is forking?

Forking creates a copy of a repository under your account to contribute without direct access.

Q37. Difference between fork and clone?

Fork is server-side copy on GitHub.
Clone is a local copy of a repository.

Q38. What are GitHub Issues?

Issues are used for bug tracking, feature requests, and task discussion.

Q39. What are GitHub Actions?

GitHub Actions automate workflows like testing, building, and deployment.

Q40. What happens when you push code?

Git transfers commits to the remote repository and updates branch pointers if possible.

SECTION E — ADVANCED / CONCEPTUAL QUESTIONS

Q41. What is Git's object database?

Git stores data as objects: blobs, trees, commits, and tags.

Q42. Why are commits immutable?

Because commit hashes depend on content; changing content changes the hash.

Q43. What is a DAG in Git?

Git history forms a Directed Acyclic Graph where commits point to parent commits.

Q44. Why is rebase dangerous on shared branches?

Because it rewrites commit history and changes hashes used by others.

Q45. How does Git ensure data integrity?

Using cryptographic hashes for all objects.

Q46. Why is staging area important in professional workflows?

It enables atomic commits and logical separation of changes.

Q47. How does Git handle large projects efficiently?

By:

- Reusing identical blobs

- Using hashes
- Compressing objects

Q48. What is a tag used for?

Tags mark specific commits, commonly for releases.

Q49. What is the difference between lightweight and annotated tags?

Annotated tags store metadata; lightweight tags are simple references.

Q50. What is the biggest Git mistake beginners make?

Not understanding the difference between local and remote history.

FINAL SUMMARY — PART 3

This section covered:

- Conceptual questions
- Practical scenarios
- GitHub workflows
- Advanced internals
- Interview traps and correct reasoning

Together with Parts 1 and 2, this forms a **complete Git & GitHub learning + interview preparation resource**.

About This Document:

This document is part of a structured learning resource series created to support clear understanding, practical learning, and independent study.

The content is designed to be:

- Structured and step-by-step
- Accessible to learners
- Suitable for self-study and revision

Usage & Sharing:

This resource may be:

- Read for personal learning
- Shared for educational purposes
- Used as reference material

Please do not modify the content structure when sharing.

Further Resources:

For future learning resources, updates, and new releases from this series:

Follow *Pranav Gujar*

Feedback & Improvements:

If you notice any errors, outdated information, or areas for improvement, feedback is encouraged through the original sharing platform.
